

RiskRadar — ML / Data Engineering Progress Update

#2

Author: Ire (ML / Data Engineer) **Covers:** Baseline model training, evaluation, and the honest limitations found

Follows: `ml-progress-update-1.md` (dataset, repo setup, 7-rule validation)

1. Overview

With feature engineering complete and the cleaned dataset saved

(`data/processed/nigerian_transactions_clean.csv` , 5,086,000 rows × 50 columns), this update covers training and evaluating the first baseline fraud detection model — a Random Forest classifier — and what we learned from it.

2. Hardware Constraint and How We Handled It

The full 5,086,000-row dataset could not be loaded or trained on directly — repeated `MemoryError` s occurred at each stage (CSV load, training) on the available machine (~7.5GB RAM).

Fixes applied, in order:

1. Loaded only the 29 columns actually needed for training (`usecols`), with explicit lightweight dtypes (`float32` / `int8` / `int32` instead of pandas' default `float64` / `int64`) — cut memory use roughly 4–6x on load.
2. Took a **stratified random sample of 1,000,000 rows** (from the full 5,086,000) for training/testing, preserving the same 3.6% fraud rate. This is a documented trade-off: training used ~20% of available data due to hardware limits, not a data-quality decision.
3. Reduced model parallelism (`n_jobs`) and tree depth to fit training within available memory.

Final working sample: 1,000,000 rows → 800,000 train / 200,000 test (stratified 80/20 split, fraud rate ~3.6% in both).

3. Baseline Model: Random Forest

Final training configuration:

```
RandomForestClassifier(  
    n_estimators=100,  
    max_depth=20,  
    min_samples_leaf=5,  
    class_weight='balanced_subsample',  
    random_state=42,  
)
```

Feature set (29 features): all 6 surviving engineered rules from Update #1 (`is_high_risk_state` , `is_high_risk_lga` , `is_ato_risk` , plus the dataset's own pre-built `spending_deviation_score` , `velocity_score` , `geospatial_velocity_anomaly` , `is_night_txn` , and related behavioral/device/velocity fields).

4. What We Found: A Real Precision Ceiling

Feature importance confirmed the model learned correctly — `is_ato_risk` (device newness) dominates at 36.5% importance, exactly matching the strong relationship found during feature validation (0% fraud rate when `new_device_transaction` is False, 4.38% when True).

However, evaluation revealed a genuine limitation, not a bug:

Threshold	Precision	Recall
0.1	4.4%	100.0%
0.3 (best F1)	4.4%	97.6%
0.5 (default)	4.3%	24.3%
0.6	100.0%	0.1%

Precision stays flat at ~4.4% across a wide range of thresholds — which is almost exactly the conditional fraud rate *within* the `is_ato_risk == 1` group we found in Update #1. **This means: once a transaction involves a new device, none of the other available features meaningfully separate real fraud from legitimate activity.** The model has correctly learned the one strong signal this dataset contains; there is no further learnable pattern within that subgroup given the current feature set.

This is consistent with our Update #1 finding that this dataset ties fraud almost entirely to device novelty — the ceiling here is a property of the data, not a modeling failure. We verified this by testing multiple configurations (different `max_depth`, `class_weight` strategies, and decision thresholds) — all converged on the same ~4.4% precision ceiling.

5. Decision: Operating Threshold and How to Use This Model

Chosen operating threshold: 0.3 — Precision 4.4%, Recall 97.6%.

Reasoning: in fraud detection, missing real fraud (false negative) is typically far costlier than an extra manual review (false positive). This threshold catches nearly all real fraud cases at the cost of a high review volume (~1 real fraud per ~23 flagged transactions).

This model should be used as a triage/flagging layer, not an auto-block system — flagged transactions should route to review, combined with the rule-engine layer (geographic risk, odd hours, velocity) the backend team is building, so multiple rule flags on the same transaction can help prioritize which flagged cases need the most urgent review.

6. Should We Get a Different Dataset?

Considered, but **not recommended at this stage** — switching datasets now would require redoing the full pipeline (augmentation, rule validation, training) with limited weeks remaining, for an uncertain improvement.

For future work / a stretch goal, the closest real-Nigerian-context alternative found is the [NIBSS Fraud Dataset on Kaggle](#) (1M NIBSS-calibrated transactions) — potentially useful as a **secondary validation set** (train on current data, test generalization on NIBSS data) rather than a replacement, if time allows after the core deliverables are done.

7. Model Artifact

Saved to `models/fraud_model_v1.pkl` (excluded from git — see `.gitignore` ; share directly with teammates who need it, or they can regenerate it by running the notebooks in order).

8. What's Next

- Integrate model output (fraud probability score at the 0.3 threshold) with the backend team's rule engine
- Consider trying XGBoost as an alternative baseline for comparison, since it may capture non-linear feature interactions differently than Random Forest
- If time allows: validate against the NIBSS dataset as a generalization check
- Build the real-time simulator (`simulator/generate.py`) out further for the live dashboard demo, including the two demo scenarios already built (impossible travel, airtime — both simulator-only, not part of the trained model)